

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 106. Adding Elements to a List

Section 1: Lecture Summary

This lecture covers **methods for adding elements to Python lists**, emphasizing the built-in list class methods: **append**, **extend**, **insert**, and **copy**. It begins with a brief explanation of Python lists as objects with associated methods. The instructor demonstrates how to add elements to lists using these methods and also introduces slicing techniques for insertion. Throughout, practical examples show the behavior and usage constraints of the methods, such as **append** accepting only one element, **extend** requiring an iterable for multiple additions, **insert** allowing insertion at arbitrary positions, and **copy** creating shallow duplicates of lists. The lecture uses Jupyter Notebook demonstrations and visual explanations to reinforce understanding.

Section 2: Key Concepts and Explanations

- Python Lists as Objects

- Lists in Python are objects created from the built-in list class.
- This class provides several **methods** to manipulate lists.

- Appending Elements

- `append(element)` adds a single element to the **end** of the list.
- Accepts exactly one element (no multiple elements).
- Can append any single object, including a list as one element.
- Example: Appending the integer `6` to list `[1,2,3,4,5]` results in `[1,2,3,4,5,6]`.

- Extending Lists

- `extend(iterable)` adds **all elements** from the given iterable to the end of the list.
- Accepts one iterable argument (like list, range, string).
- Unlike **append**, it inserts each element separately, not as a single composite element.
- Examples:

- Extending `[1,2,3,4]` with `[5,6,7]` results in `[1,2,3,4,5,6,7]`.
- Extending `[1,2]` with string `"python"` results in `[1, 2, 'p', 'y', 't', 'h', 'o', 'n']`.
- Range object like `range(5,8)` inserts elements 5,6,7.

- Inserting Elements

- `insert(index, element)` inserts a single element at the specified index in the list.
- Elements at and after the insertion index shift right.
- If index is outside valid range (too large), element is appended at the end.
- Examples:
 - Insert `50` at index `0` in `[1,2,3,4]` results in `[50,1,2,3,4]`.
 - Insert `"python"` at index `2` results in `[1, 2, 'python', 3, 4]`.
 - Index beyond last element inserts at the end.

- Using Slicing for Insertion

- Slice assignment `list[start:end] = [elements]` can insert elements.
- Example: `l1[2:2] = ` inserts `55` at index 2 without removing existing elements.
- Slicing can be used to append by slicing at the length index, e.g., `l1[len(l1):len(l1)] = [element]`.

- Copying Lists

- `copy()` creates a **shallow copy** of a list.
- The new list object is different, but the elements inside (if mutable) are referenced, not cloned.
- Changes to elements inside mutable objects affect both lists; changes to the list structure do not.
- Useful when you want a new list object but not a deep clone.

Section 3: Example Code and Use Cases

Append method: adds a single element at the end

Append multiple elements - error

`l1.append(6, 7)` `TypeError`

Append a list as a single element

Using slicing to append element

Extend method: adds all elements of an iterable at the end

Extend with string - adds each character individually

Extend with range object

Insert method: insert element at specified index

Insert at out-of-range index (inserts at end)

Using slicing to insert element at index 2

Copy method: shallow copy of a list

Modifying l8 won't affect l7 (shallow copy)

```
l1 = [1, 2, 3, 4, 5]
l1.append(6)
print(l1) # Output: [1, 2, 3, 4, 5, 6]

l1 = []
l1.append(10)
l1.append('python')
l1.append([1, 2])
print(l1) # Output: [10, 'python', [1, 2]]

l1[len(l1):len(l1)] = [20]
print(l1) # Adds 20 at the end

l2 = [1, 2, 3, 4]
l2.extend([5, 6, 7])
print(l2) # Output: [1, 2, 3, 4, 5, 6, 7]

l3 = [1, 2]
l3.extend('python')
```

```
print(l3) # Output: [1, 2, 'p', 'y', 't', 'h', 'o', 'n']
```

```
l4 = [1, 2, 3, 4]
l4.extend(range(5, 8))
print(l4) # Output: [1, 2, 3, 4, 5, 6, 7]
```

```
l5 = [1, 2, 3, 4]
l5.insert(0, 50)
print(l5) # Output: [50, 1, 2, 3, 4]
```

```
l5.insert(2, 'python')
print(l5) # Output: [50, 1, 'python', 2, 3, 4]
```

```
l5.insert(70, 'end')
print(l5) # Output: [..., 'end']
```

```
l6 = [1, 2, 3, 4]
l6[2:2] = [55]
print(l6) # Output: [1, 2, 55, 3, 4]
```

```
l7 = [1, 2, 3, 4]
l8 = l7.copy()
print(l8) # Output: [1, 2, 3, 4]
```

```
l8[0] = 100
print(l7) # Output: [1, 2, 3, 4]
print(l8) # Output: [100, 2, 3, 4]
```

Explanation:

- The code blocks demonstrate how to add elements using `append`, `extend`, and `insert`.
- Using slicing operations to insert or append elements is shown as an alternative.
- The `copy` method's shallow copy behavior is demonstrated with list modifications.

Section 4: Key Takeaways

- Lists in Python are objects with built-in methods for element addition.
- **append(element)** adds a single element at the end of the list.
- **extend(iterable)** adds each element from an iterable to the end of the list.
- **insert(index, element)** inserts an element at a specific index, shifting subsequent elements.

- You can use **slice assignment** to insert or append elements by replacing empty slices.
- The **copy()** method creates a shallow copy of a list — a new list object but references the same elements.
- Understanding these methods allows flexible list manipulation during programming.
- Append only takes one argument, while extend expects an iterable.
- Insert allows insertion at any index, including out-of-range (which adds at end).
- Strings, ranges, and other iterables can be used with extend for bulk addition.